ReadRight 2.0

Part 3 - Extension Proposal

Mihir Patel & Mir Patel

This proposal outlines how we plan to extend Project 2 (Jaredburne47/4150ReadRight) into ReadRight 2.0. It is organized around the three required pillars: speech/pronunciation, AI story builder, and Dolch sight-word games.

Pillar 1 - Speech & Pronunciation

What We're Inheriting

- A working Azure Pronunciation Assessment integration that sends real audio to Microsoft's API and returns per-word accuracy scores.
- A broken offline fallback (cloud_fallback_assessor.dart) that silently returns a zero score when Azure is unavailable, which we identified as a Priority 1 fix in our Part 1 evaluation.
- A hardcoded 3-second recording window that doesn't account for word length, and leftover debug print statements in production service files.

Planned Improvements

- **Fix the fallback assessor.** Replace the hardcoded zero return in cloud_fallback_assessor.dart with a real local fallback which would either be a string-similarity score similar to Project 1's approach or a 'try again later' UI state that is honest with the student rather than falsely marking them incorrect.
- **Adaptive recording duration.** Replace the hardcoded 3-second cap with a dynamic window that scales to the number of syllables in the target word, or uses silence detection to auto-stop the recording. A single-syllable word like 'the' needs far less time than 'because'.
- **Surface per-word accuracy to students.** Azure's API already returns per-word and per-phoneme accuracy scores inside the NBest response, this data is captured in AssessmentResult but never shown to the student. We plan to display a simple visual breakdown so a child can see which part of the word they struggled with, not just a pass/fail result.
- **Remove debug print statements.** Replace all production print() calls in the speech service layer with a proper logging abstraction.

Educational Value

- Fixing the fallback means students practicing with a weak connection no longer receive false failure feedback, which directly protects the confidence-building loop that mastery-based literacy learning depends on.
- Adaptive recording duration ensures the app captures a complete pronunciation attempt for every word. Longer words currently risk being cut off, which produces inaccurate scores that could mislead both the student and their teacher.
- Surfacing per-word accuracy gives students and teachers a specific, actionable signal ('you struggled with the ending of this word') rather than a pass/fail, which is more aligned with how phonics instruction actually works.

Pillar 2 - AI Story Builder

Planned Features

- **Teacher-initiated generation.** Teachers will trigger story generation from the teacher dashboard by selecting a Dolch word list and optionally a theme. The student never directly interacts with the AI.
- **Short, readable stories.** Each generated story will be 3–5 sentences long, written at the appropriate reading level for the selected Dolch list, and will naturally include as many of the target sight words as possible. Stories will be displayed one sentence at a time in the student view.
- **Word highlighting.** Dolch sight words will be highlighted in the displayed story so students can identify and practice them in context, reinforcing the connection between isolated word practice and real reading.

AI Integration Strategy

Pillar 3 - Dolch Sight-Word Games

Planned Game

We plan to build two Dolch sight-word game. It uses the same underlying word-list and mastery data model already present in the inherited codebase.

Game 1 - Fill the Blank

- How it works: A short sentence appears on screen with one sight word missing, represented by a blank. The student taps the correct word from 3–4 choices displayed below the sentence to complete it. Correct answers advance the mastery counter; incorrect answers flag the word for additional review.
- Educational value: Reinforces sight words in grammatical context, and the student has to understand which word makes the sentence make sense, not just recognize the word in isolation. This bridges word recognition to reading comprehension, a higher-order literacy skill that isolated flashcard practice doesn't address.

Game 2 — Flash Dash

- **How it works:** A sight word appears on screen for 2 seconds, then disappears. The student then selects the word they just saw from three choices displayed below. Correct answers advance the mastery counter; incorrect answers flag the word for additional review. Missed words recycle back into the round until the student clears them all.
- **Educational value:** Trains rapid visual recognition of whole words, which is how fluent readers actually process sight words, not by sounding them out, but by recognizing the whole shape instantly. This addresses the fluency component of reading beyond just reading.

Technical Approach:

State Management

- We will keep the existing Provider-based state management pattern from the inherited codebase. Each new feature (story builder, each game) will get its own ChangeNotifier provider rather than expanding existing ones, keeping concerns separated.
- Game state (current word, score, timer) will be local to each game's provider and not persisted between sessions, only the mastery outcome (did the student get this word right) will be written to the existing AttemptRecord model.

Data Model Direction

- We will extend the existing Student and AttemptRecord models to include a source field indicating which activity generated the attempt (pronunciation practice, Fill the Blank, or story reading). This lets the teacher dashboard break down a student's progress by activity type without restructuring the model.
- Generated stories will be stored as a new StoryRecord model in Firestore (one document per generated story, linked to the ClassRoom that generated it). Stories are teacher owned content, not student content, so they live at the classroom level rather than the student level.

Provider Plan for Speech

- The existing PronunciationAssessor interface will be kept as-is. We will replace the body of CloudFallbackAssessor with a real implementation rather than creating a new class, so the rest of the app's wiring doesn't change.
- Recording duration will be controlled by a new WordTimingService that takes a word string and returns a recommended recording duration in milliseconds, based on syllable count. This service will be injected into the existing SpeechService rather than modifying SpeechService's interface.

Major Architecture Changes Anticipated

- **New: backend proxy for AI API calls.** A Firebase Cloud Function will sit between the Flutter app and the Anthropic API. The app sends a word list and theme; the function calls Claude and returns the story. This keeps the API key server-side.
- **New: game navigation tab.** The student navigation bar will gain a Games tab alongside the existing Practice tab. The Fill the Blank game is accessible from this tab.
- **New: story viewer screen.** A new StudentStoryScreen will display teacher-generated stories sentence by sentence with Dolch words highlighted. Navigation to this screen will appear in the student home after a teacher has generated at least one story for their class.
- **No changes to:** the existing auth flow, the teacher dashboard's class and student management screens, the accessibility service, or the Firestore data structure for Student and ClassRoom.